

Oh My Zsh: A Practical Guide to Your Shell

Mick Cooney — June 2026

Table of Contents

Oh My Zsh: A Practical Guide to Your Shell	2
Introduction	2
What This Primer Covers	2
What This Primer Is Not	3
Assumed Background	3
Why ZSH Over Bash	3
Completion	3
Globbing	4
Parameter Expansion	4
History	4
Named Directories	5
How Oh My Zsh Works	5
The lib/ Layer	6
Enabling and Disabling Plugins	6
Plugins Worth Knowing Deeply	6
git	7
fzf	7
zsh-autosuggestions	8
zsh-syntax-highlighting	8
history	9
sudo	9
extract	9
colored-man-pages	9
command-not-found	9
timer	10
rsync	10
systemd	10
mise	10
gh	11
Themes: What a Good Prompt Does	11
What Information Belongs in a Prompt	11
robbyrussell	11
agnoster	11
Powerlevel10k (p10k)	12

Features Most OMZ Users Never Find	12
Custom Aliases and Functions in <code>custom/</code>	12
The <code>omz</code> Command	13
Smart Directory Navigation Without <code>cd</code>	13
<code>z</code> — Smarter Directory Jumping	14
ZSH Suffix Aliases	14
Global Aliases	14
ZSH Line Editor (ZLE) Widgets	15
Your Setup: An Honest Audit	15
What You Have	15
What’s Missing	16
Things You’re Probably Underusing	16
Small Additions Worth Considering	17
When OMZ Is Not Enough	17
Quick Reference: What to Do Next	18
Further Reading	18

Oh My Zsh: A Practical Guide to Your Shell

Mick Cooney — June 2026

Introduction

Most people who install Oh My Zsh do the same three things: change the theme, add the `git` plugin, and then forget about it for two years. The shell gets faster and prettier, but the actual productivity gain stays shallow. The plugin list grows by one every time someone on Reddit says “you need X,” but the full picture of what the framework can do — and what ZSH itself can do underneath it — never quite lands.

This primer tries to fix that. It starts with why ZSH is genuinely different from Bash (not just cosmetically), explains how Oh My Zsh is actually structured, goes deep on the plugins and features that change how you work, and ends with a concrete look at a real setup: what’s good, what’s missing, and what to do about it.

What This Primer Covers

- What ZSH adds over Bash that actually matters in daily use
- How Oh My Zsh works under the hood — plugins, themes, the `custom/` directory
- The plugins worth knowing deeply, with real examples of what they do
- Themes: what a good prompt gives you and when to switch

- Advanced ZSH features that most OMZ users never touch
- A concrete audit of an existing setup with actionable recommendations

What This Primer Is Not

This is not a Bash-to-ZSH migration guide, and it does not cover every one of the three hundred-plus plugins in the OMZ repository. It also does not cover Prezto, Fish, or Nushell — all of which are worth knowing about, but that’s a different document. The focus is on getting maximum value out of Oh My Zsh specifically, for someone who already has it installed and wants to stop leaving capability on the table.

Assumed Background

You are comfortable in a terminal, have used Bash or ZSH before, and already have Oh My Zsh installed. You know what a `.zshrc` is. You don’t need to know anything about ZSH internals.

Why ZSH Over Bash

It is worth spending a moment on why ZSH exists and what it actually adds, because understanding the foundation makes the Oh My Zsh layer easier to appreciate. ZSH is not Bash with a better theme engine. It has genuinely different capabilities.

Completion

ZSH’s tab completion system — called `compsys` — is architecturally different from Bash’s. In Bash, completion is mostly prefix matching: you type `git com` and it completes to `git commit`. In ZSH, completion is programmable and contextual. It knows what `git commit` accepts as flags. It knows that `-m` takes a string. It can complete hostnames from your SSH config, Docker container names, systemd unit names, `man` page sections, and hundreds of other context-specific things. Plugin authors write completion functions, and OMZ ships many of them.

You see this in practice every time you type `git <Tab>` and get a menu with descriptions, or type `docker run --<Tab>` and get the full flag list. Bash can approximate some of this with `bash-completion`, but ZSH’s system is deeper and more consistent.

Globbering

ZSH's glob system is significantly more powerful than Bash's. This is not a feature most people discover organically, but once you know it, you reach for it constantly.

*# Match all .log files recursively (** is a ZSH glob, not Bash)*

```
ls **/*.log
```

Only files (not directories) modified in the last day

```
ls *(m-1.)
```

Only executable files

```
ls *(x)
```

Files larger than 1MB

```
ls *(Lm+1)
```

Combine: executable files modified in the last week

```
ls *(x,m-7)
```

The glob qualifiers (the characters inside (...)) filter on file metadata: type, modification time, size, permissions. This is the kind of thing you'd normally pipe through `find` with four flags; in ZSH it's a glob modifier.

Parameter Expansion

ZSH extends Bash's already capable parameter expansion with additional flags:

```
path="/home/mcooney/workspace/ai_assisted_research/ohmyzsh_primer/ohmyzsh_primer.md"
```

Bash-compatible: get filename without path

```
echo ${path##*/}          # ohmyzsh_primer.md
```

ZSH-only: split on delimiter and get last element

```
echo ${path:t}            # ohmyzsh_primer.md
```

Strip extension

```
echo ${path:t:r}          # ohmyzsh_primer
```

Directory part

```
echo ${path:h}            # /home/mcooney/workspace/ai_assisted_research/ohmyzsh_primer
```

Uppercase

```
str="hello world"
```

```
echo ${(U)str}            # HELLO WORLD
```

Split into array on /

```
parts=(${(s:/:)path})
```

```
echo $parts[2]            # home
```

The (U), (L), (s:delim:) and similar parameter flags are ZSH-specific. They're not a party trick; they eliminate a lot of `awk/sed` one-liners for string manipulation in scripts.

History

ZSH history is more configurable than Bash's. The key options that actually matter:

```

HISTSIZE=100000      # lines in memory
SAVEHIST=100000      # lines on disk
setopt HIST_IGNORE_DUPS # don't record adjacent duplicates
setopt HIST_IGNORE_SPACE # don't record commands starting with space
setopt SHARE_HISTORY   # share history across sessions in real time
setopt EXTENDED_HISTORY # record timestamp with each command

```

SHARE_HISTORY means that if you have three terminals open, history flows between them as you type — you don't have to close and reopen to pick up commands from another session. HIST_IGNORE_SPACE is the one you want for sensitive commands: prefix any command with a space and it doesn't go into history.

Named Directories

ZSH lets you name directories and use them like variables in paths:

```

hash -d work=/home/mcooney/workspace
hash -d proj=/home/mcooney/workspace/ai_assisted_research

# Now you can use ~work and ~proj anywhere
cd ~proj
ls ~work/some_other_thing

```

Put these in your .zshrc and they persist. It's the equivalent of shell aliases for paths, except they work in any context where a path is accepted.

How Oh My Zsh Works

Understanding the structure of OMZ makes it easier to know what to change and where. The repository lives at ~/.oh-my-zsh/ and has a clear layout:

```

~/.oh-my-zsh/
lib/          # Core ZSH configuration loaded at startup
plugins/      # Bundled plugins (git, docker, fzf, etc.)
themes/       # Bundled themes
custom/       # Your overrides: themes, plugins, functions
  plugins/    # Third-party or custom plugins go here
  themes/     # Custom themes go here
  aliases.zsh # Put custom aliases here
  functions.zsh # Put custom functions here (or any .zsh file)
templates/    # Template for new .zshrc
tools/        # Install/update scripts

```

When your shell starts, OMZ loads oh-my-zsh.sh, which: 1. Loads everything in lib/ (history config, completion setup, key bindings, utility functions) 2. Sources each enabled plugin's .plugin.zsh file 3. Loads the theme 4. Sources everything in custom/ that ends in .zsh

The custom/ directory is where your personalizations live and where they survive OMZ updates. If you write a custom function and put it in ~/.oh-my-zsh/custom/myconfig.zsh, it

gets loaded automatically. If you put the same function in a bundled plugin file, the next `omz update` will overwrite it.

The `lib/` Layer

Most users never look at `lib/`, but it does a lot. The files there configure:

- `completion.zsh` — sets up `compsys`, enables menu completion, configures case-insensitive matching
- `history.zsh` — the history options described earlier
- `key-bindings.zsh` — registers `Ctrl+R` for history search, `Ctrl+A/E` for line navigation, etc.
- `git.zsh` — the helper functions used by git-aware themes
- `functions.zsh` — utility functions used internally, like `omz_urlencode`

You rarely edit these directly, but knowing they exist explains why certain behaviors appear without you configuring them.

Enabling and Disabling Plugins

The `plugins` array in `.zshrc` is straightforward:

```
plugins=(  
  git  
  docker  
  fzf  
  zsh-autosuggestions  
)
```

Each name must match either a directory in `~/.oh-my-zsh/plugins/` or `~/.oh-my-zsh/custom/plugins/`. OMZ looks in both places. Third-party plugins (like `zsh-autosuggestions`) are cloned into `custom/plugins/` and added to the list.

The startup cost of plugins is real. Each plugin sources a file. Most are small, but a plugin that initializes a large tool (like a package manager) adds perceptible latency. Keep only what you actually use.

Plugins Worth Knowing Deeply

This is not a list of every plugin. It is a deep look at the ones that change how you work, with concrete examples of what they actually do.

git

The `git` plugin ships over 150 aliases. Most people use `gst` (`git status`) and `gaa` (`git add --all`) and call it done. The more interesting ones:

```
# Commonly used
gst  # git status
ga   # git add
gaa  # git add --all
gcmgs # git commit -m
gp   # git push
gl   # git pull
gco  # git checkout
gcb  # git checkout -b
gd   # git diff
gds  # git diff --staged
glog  # git log --oneline --decorate --graph

# Less obvious but very useful
gupa  # git pull --rebase --autostash
gcf   # git config --list
gclean # git clean -id (interactive clean)
gwip  # commit a work-in-progress stash
gunwip # undo the last WIP stash commit
```

The `gwip` / `gunwip` pair is genuinely useful when you need to quickly switch branches mid-work without a full commit. `gwip` creates a temporary [WIP] commit; `gunwip` undoes it and restores the working state.

The plugin also provides `git_prompt_info()`, which most themes use to show the current branch and status in the prompt.

fzf

`fzf` is a fuzzy finder for the terminal. The OMZ plugin wires it into three key bindings:

```
Ctrl+R  - fuzzy search your command history
Ctrl+T  - fuzzy find a file in the current directory tree
Alt+C   - fuzzy cd into a subdirectory
```

Most people know `Ctrl+R` for history. The other two are underused. `Ctrl+T` is faster than typing a path when you remember the filename but not the full path. `Alt+C` is faster than `cd` with tab completion when you're navigating several levels deep.

`fzf` also integrates with commands directly:

```
# Preview files while browsing
fzf --preview 'cat {}'

# Kill a process by fuzzy-searching running processes
kill -9 $(ps aux | fzf | awk '{print $2}')

# Open a recent git branch
git checkout $(git branch | fzf)

# Select from command history and re-run
eval $(history | fzf | awk '{$1=""; print}')
```

The underlying tool accepts any input on stdin and returns the selected line on stdout, so it composes with everything. The OMZ plugin just provides the shell integration; learning `fzf`'s flags unlocks the full power.

zsh-autosuggestions

This plugin shows a greyed-out suggestion as you type, based on your history. Press the right arrow key to accept the full suggestion, or press `End` to move to the end of the current word.

You type:

```
git commit
```

It shows (in grey):

```
git commit -m "fix typo in README" ← suggestion from history
```

Right arrow accepts the whole thing

Ctrl+F accepts one character at a time

The plugin also supports accepting a suggestion word-by-word. By default this uses `Ctrl+Space` but can be rebound:

```
bindkey '^ ' autosuggest-accept-word
```

One tuning option worth knowing: by default it only suggests from history. You can also enable completion-based suggestions:

```
ZSH_AUTOSUGGEST_STRATEGY=(history completion)
```

This means if history has no match, it falls back to ZSH completions.

zsh-syntax-highlighting

This plugin is not in the default OMZ bundle — it requires a separate install. It is, however, one of the highest-value things you can add. It highlights commands in real time as you type:

- Commands that exist: green
- Commands that don't exist: red
- Strings and quoted arguments: yellow
- Options/flags: light blue
- Pipes, redirects, semicolons: magenta

The value is catching typos before you press `Enter`. You type `pythno script.py` and the red highlight catches it immediately. You type `ls --recusrive` and it's red before you hit `Enter`. After using it for a week you will miss it in any shell that doesn't have it.

Install:

```
git clone https://github.com/zsh-users/zsh-syntax-highlighting.git \
  ${ZSH_CUSTOM:- ~/.oh-my-zsh/custom}/plugins/zsh-syntax-highlighting
```

Then add `zsh-syntax-highlighting` to your `plugins` array. It must be **last in the list** — this is important; the plugin works by modifying the line buffer, and it needs to run after everything else.

history

The `history` plugin provides convenient aliases:

```
h          # full history (with timestamps if EXTENDED_HISTORY is set)
hs         # history | grep (case-sensitive)
hsi        # history | grep (case-insensitive)
# Find all git commands you've run
hs git

# Find recent docker run commands
hsi "docker run"
```

Simple but far faster than typing `history | grep` every time.

sudo

Press `Esc Esc` (two taps) to prepend `sudo` to the current command line or re-run the last command with `sudo`. No plugin is simpler or more immediately useful.

```
# You run:
apt install something-big
# Permission denied.

# Press Esc Esc
# Line becomes:
sudo apt install something-big
```

extract

The `extract` plugin provides a single command `x` (or `extract`) that works on any archive format:

```
x archive.tar.gz
x archive.zip
x archive.bz2
x archive.7z
x archive.tar.xz
x archive.rar
```

You never need to remember whether it's `-xzf` or `-xjf` or `unzip` or `7z x`. The plugin detects the format and uses the right tool. It works as long as the relevant tool is installed on the system.

colored-man-pages

Makes `man` pages render with color instead of monochrome. The improvement is significant for readability, especially for long man pages with lots of structure. No configuration needed; just include it in your plugins list.

command-not-found

When you type a command that doesn't exist, instead of a bare "command not found" error, the shell suggests the package you need to install:

```
$ tree
zsh: command not found: tree
```

Command 'tree' not found, but can be installed with:
sudo apt install tree

Requires `command-not-found` to be installed on the system (most Ubuntu/Debian systems have it). Saves a round-trip to the package manager search.

timer

Shows how long each command took, appended to the right side of your prompt after the command completes. Useful for scripts and builds where you want a rough sense of runtime without explicitly using `time`.

```
some-long-command [12s]
```

No configuration needed. If you find it noisy for short commands, set a minimum threshold:
`TIMER_THRESHOLD=2` # only show if command took > 2 seconds
`TIMER_FORMAT='%d seconds'`

rsync

Provides aliases for common `rsync` patterns:

```
rsync-copy      # rsync -avz (archive, verbose, compress)
rsync-move      # rsync -avz --remove-source-files
rsync-update    # rsync -avzu (skip files newer at destination)
rsync-synchronize # rsync -avzu --delete (full sync)
```

The flags are ones you'd reach for often but never quite remember the exact combination of.

systemd

Provides short aliases for common `systemctl` and `journalctl` operations:

```
sc-status # systemctl status
sc-start  # systemctl start
sc-stop   # systemctl stop
sc-restart # systemctl restart
sc-enable # systemctl enable --now
sc-disable # systemctl disable --now
sc-list   # systemctl list-units
jc        # journalctl -xe (recent logs with context)
jcf       # journalctl -f (follow/tail logs)
```

If you interact with `systemd` services regularly, these cut down the typing considerably.

mise

The `mise` plugin initializes `mise` (the polyglot version manager, formerly `rtx`) and enables its completions. It runs `mise activate zsh` during shell startup so that tool versions are correctly set in every shell session. Without this, you'd need to call `eval "$(mise activate zsh)"` manually in your `.zshrc`.

gh

Completions for the GitHub CLI (gh). Tab-completion for commands, flags, and — notably — repository names and PR/issue numbers when the context makes them available. If you use gh frequently, this saves a lot of flag-hunting.

Themes: What a Good Prompt Does

The theme controls what your prompt looks like. Most people pick one that looks appealing and stop there. But a well-designed prompt is actually functional: it puts the right information in front of you at the right time.

What Information Belongs in a Prompt

A practical prompt should show:

1. **Current directory** — always. Abbreviated to the last 2-3 components if deep.
2. **Git state** — current branch, whether there are staged changes, unstaged changes, untracked files, and whether the branch is ahead/behind the remote.
3. **Last command exit code** — some indication of whether the last command succeeded or failed.
4. **Optionally:** active virtual environment, tool versions, execution time, user@host for remote sessions.

What a prompt should not do: show things that never change (always the same user on the same machine), show timestamps on every line (the `timer` plugin handles that better), or take noticeable time to render.

robbyrussell

The default theme. Clean, minimal:

```
→ ai_assisted_research git:(main) ✕
```

Shows directory and git branch. The `✕` indicates there are uncommitted changes. It's good for its simplicity, but it doesn't show whether changes are staged vs unstaged, doesn't show remote divergence, and gives no indication of exit status.

agnoster

A powerline-style theme that uses special Unicode characters to create a segmented prompt:

```
mcooney@hostname ~/workspace main ✕
```

(The segments have colored backgrounds in the actual terminal.) Shows user, host, path, git branch, and git state. More information than robbyrussell. Requires a Nerd Font or Powerline-patched font; without one, the segment characters render as boxes.

The `agnoster-timestamp-newline` theme in your custom themes directory is a variant that adds a timestamp and puts the command input on its own line, which makes long paths more readable.

Powerlevel10k (p10k)

Powerlevel10k is the current standard for people who want a serious prompt. It is not bundled with OMZ but is the most popular third-party theme by a significant margin.

What makes p10k different:

- **Configuration wizard** — runs `p10k configure` and walks you through every prompt option interactively, showing previews. You end up with a `.p10k.zsh` config that captures exactly what you chose.
- **Instant prompt** — p10k renders a partial prompt immediately, before the shell finishes loading. Shell startup feels instant even with slow plugins.
- **Async segments** — git status and other slow segments render asynchronously so they don't block input.
- **Transient prompt** — old prompts in the scroll buffer collapse to a minimal form, keeping the terminal cleaner.

For a full git-aware, multi-context prompt (git state, virtualenv, node version, exit status, command timing) without visible startup lag, p10k is the practical choice.

Install:

```
git clone --depth=1 https://github.com/romkatv/powerlevel10k.git \
  ${ZSH_CUSTOM:-$HOME/.oh-my-zsh/custom}/themes/powerlevel10k
```

Then set `ZSH_THEME="powerlevel10k/powerlevel10k"` and run `p10k configure`.

Features Most OMZ Users Never Find

Custom Aliases and Functions in `custom/`

Any `.zsh` file in `~/.oh-my-zsh/custom/` is sourced automatically. This means you don't need to clutter `.zshrc` with personal aliases — put them in `~/.oh-my-zsh/custom/aliases.zsh` and they load automatically and survive OMZ updates:

```
# ~/.oh-my-zsh/custom/aliases.zsh
```

```
alias ll='ls -lAh --color=auto'
alias ports='ss -tlnp'
alias myip='curl -s ifconfig.me'
alias reload='source ~/.zshrc'
alias ...='cd ../../'
alias ....='cd ../../../'
```

Functions go in the same place or their own file:

```
# ~/.oh-my-zsh/custom/functions.zsh
```

```
# Create directory and cd into it
mcd() { mkdir -p "$1" && cd "$1" }
```

```
# Quick find by name
f() { find . -name "$1*" 2>/dev/null }
```

```
# Show the last N lines of a log file with color
lf() { tail -f "${1:-/var/log/syslog}" | grep --color=auto -E "ERROR|WARN|INFO|$" }
```

The omz Command

OMZ ships a management command that most users don't know exists:

```
omz update           # Update oh-my-zsh
omz plugin list       # List all available plugins
omz plugin enable git # Enable a plugin without editing .zshrc
omz plugin disable git # Disable a plugin
omz theme list        # List available themes
omz theme use agnoster # Switch theme temporarily
omz changelog         # Show recent OMZ changes
```

The plugin list output is useful for discovery — there are plugins for tools you use that you may not know OMZ ships.

Smart Directory Navigation Without cd

ZSH can change directories without typing cd, if you enable it:

```
setopt AUTO_CD
```

With this set, typing a directory name alone (without cd) changes into it:

```
# Without AUTO_CD:
cd ~/workspace/ai_assisted_research
```

```
# With AUTO_CD:
~/workspace/ai_assisted_research
# or if it's in CDPATH:
ai_assisted_research
```

Combine with CDPATH to jump to frequently used directories:

```
CDPATH=.:~/workspace
```

With this set, typing ai_assisted_research from anywhere searches ., ~, and ~/workspace for that directory name and changes into it if found.

z — Smarter Directory Jumping

The `z` plugin (bundled with OMZ) tracks which directories you visit and lets you jump to them by partial name, ranked by how often and how recently you've been there:

```
# After you've visited ~/workspace/ai_assisted_research a few times:
z ai_assisted # cd's to ~/workspace/ai_assisted_research
z research    # same

# It picks the highest-ranked match for the pattern
z num         # might jump to ~/workspace/ai_assisted_research/numerical_analysis_primer
```

The ranking function (“frecency”) is frequency × recency — directories you go to often and recently rank highest. For codebases you navigate daily, `z` eliminates most explicit `cd` commands.

`zoxide` is a faster modern alternative with the same concept. It integrates with OMZ via a plugin and can be configured to replace `cd` entirely:

```
# Install zoxide, then add to .zshrc:
eval "$(zoxide init zsh --cmd cd)"
# Now 'cd' uses zoxide's frecency ranking
```

ZSH Suffix Aliases

ZSH supports suffix aliases — aliases that apply to the file extension, so the shell knows what program to use to open a file type:

```
alias -s py=python3
alias -s md=glow      # terminal markdown renderer
alias -s pdf=evince
alias -s html=firefox
```

With these, typing `script.py` at the prompt opens it with `python3`, `README.md` opens with `glow`, and so on. You don't need to prefix the program name.

Global Aliases

Regular aliases only work at the start of a command. Global aliases work anywhere in a command line:

```
alias -g G='| grep'
alias -g L='| less'
alias -g T='| tail -f'
alias -g NE='2>/dev/null'
alias -g NUL='>/dev/null 2>&1'
# With global aliases:
ps aux G python      # ps aux | grep python
cat big_log T        # cat big_log | tail -f
some-command NUL     # some-command >/dev/null 2>&1
```

This is a ZSH-specific feature with no Bash equivalent.

ZSH Line Editor (ZLE) Widgets

ZSH's line editor is programmable. You can write functions and bind them to key sequences:

Paste last argument of last command (already bound to Alt+. by default)

But you can write custom widgets:

Widget: surround current word in quotes

```
quote-word() {  
  zle select-in-word  
  LBUFFER="${LBUFFER}\ "  
  RBUFFER="\ "${RBUFFER}  
}  
zle -N quote-word  
bindkey '^[q' quote-word # Alt+Q
```

Widget: push current command to a stack, run something else, pop back

This is already provided by ZSH as push-line (Ctrl+Q by default)

Type a command, realize you need to run something first, press Ctrl+Q

Run the other command, press Enter, your original command is restored

push-line (bound to Ctrl+Q in OMZ) is worth knowing: if you're mid-command and realize you need to run something else first, Ctrl+Q parks your current input, you run what you need, and your original command comes back on the next prompt.

Your Setup: An Honest Audit

Here is a concrete look at what is active in your configuration, what is working well, and where the gaps are.

What You Have

Your active plugins:

```
git ssh-agent colored-man-pages command-not-found extract history  
docker timer sudo gh podman systemd chezmoi rsync fzf  
mise zsh-autosuggestions zsh-clause-code-shell
```

This is a well-considered list. The timer, sudo, extract, history, and colored-man-pages plugins are all underrated workhorses that most setups skip. You have fzf and zsh-autosuggestions, which are the two highest-value plugins. The tool-specific plugins (docker, podman, systemd, gh, chezmoi, rsync, mise) are all appropriate for someone with your tooling.

Your history config is solid: 100k entries, eternal history logging, session sharing. Most people have 1000 entries and wonder why they can't find old commands.

What's Missing

zsh-syntax-highlighting — install this first.

This is the gap that will have the most immediate impact. Every command you type is colour-coded as you type it: green means the command exists, red means it doesn't. You'll catch typos before Enter, see unclosed quotes highlighted, and generally have a much more readable command line. It's off by default because it's a third-party plugin, but it's nearly universal in well-configured ZSH setups.

```
git clone https://github.com/zsh-users/zsh-syntax-highlighting.git \
  ~/.oh-my-zsh/custom/plugins/zsh-syntax-highlighting
```

Add **zsh-syntax-highlighting** to your plugins list, and — critically — put it **last**:

```
plugins=(
  ...your existing plugins...
  zsh-syntax-highlighting # must be last
)
```

z for directory jumping.

You're navigating a workspace with a dozen active directories. The **z** plugin is bundled with OMZ — just add it to your plugins list. After a few days of normal use, **z research** or **z building** gets you directly to the right directory from anywhere. No install needed:

```
plugins=(... z ...)
```

Consider your theme.

You have **agnoster-timestamp-newline** in your custom themes but are using **robbyrussell**. This may be intentional, but it's worth trying the custom theme if you haven't recently — the newline variant is easier to read when you're deep in a long path. If you want more git state visibility (staged vs unstaged vs untracked vs ahead/behind remote), **p10k** gives you all of that with a five-minute setup via **p10k configure**.

Things You're Probably Underusing

fzf beyond Ctrl+R.

You have **fzf** installed. Ctrl+R for history search is probably the only thing you use it for. Try: - Ctrl+T while typing a command to fuzzy-find a file argument - Alt+C to fuzzy-jump into a subdirectory

Both of these are wired up by the **fzf** OMZ plugin and work out of the box.

The **sudo** plugin's Esc Esc.

If you're not already using this reflex, it eliminates one of the most common terminal annoyances: running a command, getting "permission denied," and having to retype it with **sudo**. Press Esc Esc after the failed command and it prepends **sudo** for you.

extract / the **x** command.

If you're still typing `tar -xzf` or looking up whether it's `-xjf` for `bz2`, the `extract` plugin's `x` command handles every archive format uniformly. You already have the plugin; just use `x archive.tar.gz` instead of remembering flags.

History grep.

The `history` plugin gives you `hs` and `hsi` for grepping history. If you're typing `history | grep docker` to find that run command from last week, `hs docker` does the same thing in fewer keystrokes.

Small Additions Worth Considering

A few things that are low-effort and pay off quickly:

```
# In ~/.oh-my-zsh/custom/aliases.zsh

# Directory navigation shortcuts
alias ..='cd ..'
alias ...='cd ../../'
alias ....='cd ../../..'

# Quick list
alias ll='ls -lAh --color=auto'

# Git shortcuts beyond what the git plugin provides
alias gst='git status' # already from git plugin, but worth knowing

# Show port usage
alias ports='ss -tlnp'
```

And a few ZSH options worth adding to your `.zshrc` if they're not there:

```
setopt AUTO_CD      # type a directory name to cd into it
setopt CORRECT      # suggest corrections for mistyped commands
setopt HIST_IGNORE_SPACE # commands starting with space aren't recorded
```

When OMZ Is Not Enough

Oh My Zsh is the right choice for most people, but there are situations where it starts to show its age.

Startup speed. OMZ adds measurable startup latency, mostly from sourcing plugins and initializing completions. On modern hardware this is usually under 200ms, which is imperceptible. But if you're on a slow machine, opening many terminals, or doing a lot of `zsh -c` invocations in scripts, it accumulates. `zinit` and `zplug` are plugin managers that load plugins lazily or in parallel and can reduce startup time significantly. `sheldon` is another option with a simple TOML configuration.

Opinionated defaults. OMZ makes choices about completion settings, history handling, and key bindings that most people are fine with. If you find yourself fighting the defaults, vanilla ZSH with `zinit` and hand-picked plugins gives you more control.

Alternatives worth knowing. Prezto is OMZ's more modular cousin — faster startup, more configurable, smaller default footprint. Fish shell takes a different approach entirely: better out-of-the-box experience, but incompatible syntax with Bash/ZSH, which matters if you write a lot of scripts. Nushell is the most radical departure — structured data instead of text streams, genuinely different mental model, not a daily driver for most people yet.

For the typical developer workflow, OMZ with a curated plugin list and `p10k` as the theme is a well-tested configuration that will serve you well without requiring you to build from scratch.

Quick Reference: What to Do Next

In order of impact:

1. **Install `zsh-syntax-highlighting`** — see the install command in the plugin section above. Add it last in your plugins list.
2. **Add `z`** to your plugins list (no install needed, it's bundled). Use it for a week.
3. **Try `Ctrl+T` and `Alt+C`** with `fzf` — fuzzy file and directory completion. They're already wired up.
4. **Internalize `Esc Esc`** (`sudo` plugin) and `x` (extract plugin) — you have both, just build the habit.
5. **Consider `p10k`** if you want a richer prompt with git state detail.
6. **Read `omz plugin list`** — there may be completions for tools you use daily that you didn't know OMZ ships.

Further Reading

- **OMZ repository** — `~/.oh-my-zsh/` itself: `plugins/<name>/<name>.plugin.zsh` files are the source of truth for what each plugin does and what commands/aliases it provides.
- **ZSH manual** — `man zsh` is the definitive reference. The sections on parameter expansion, globbing qualifiers, and ZLE are particularly worth having open.

- **Powerlevel10k** — the configuration wizard at `p10k configure` is unusually good documentation in interactive form.
- **ZSH Users mailing list** / `r/zsh` — for real-world configuration questions; the OMZ wiki has gotten stale in places.